

MEI/MCA/MIA/MMC

Computação Paralela

Introduction to Parallelism

2026/27

João Luís Sobral

Revisão da última aula (1)

...urse: performance engineering

Main goal

- Learn how to build programs that run efficiently on modern computers
  - Understand the key performance challenges
- The only way to get there:
  - build programs that do lots of independent things in parallel

1st big challenge

Why parallelism?

- Sequential performance stopped improving around 2000
- New performance improvements come mostly from more parallelism
- New programs (code) are needed specially for large scale parallelism

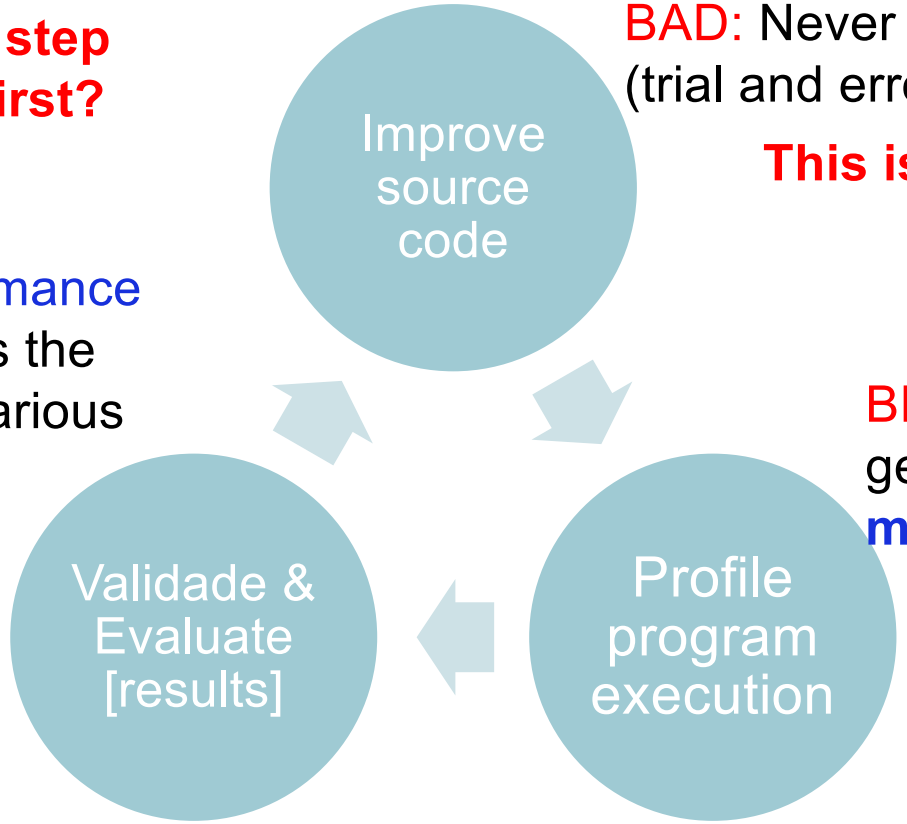
Revisão da última aula (2)

ce optimization

- **Typical optimization cycle of a program**

Note: optimization also includes exploitation of parallelism

Which step to do first?



BAD: Never start from here (trial and error approach)

This is guessing!

BEST: use performance **models** to assess the performance of various alternatives

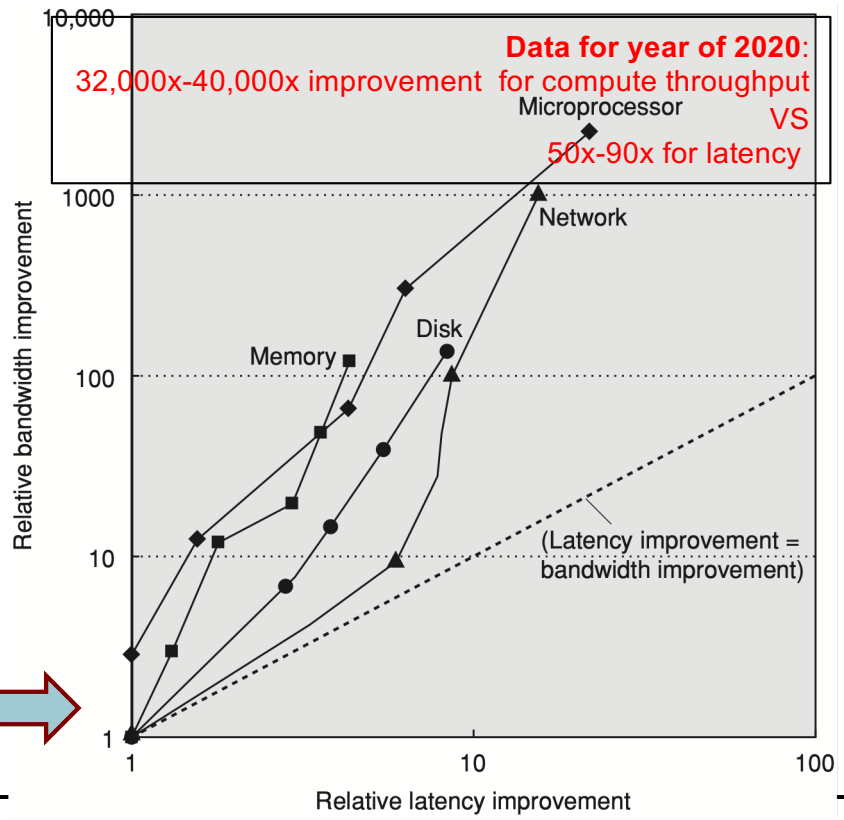
BETTER: start by getting performance **metrics**

Big challenge:
Benchmarking!
- What to measure?
- How to measure?

Typical performance metrics

- **Latency** (or response time)
  - time required to perform an operation (e.g. program execution time)
  - **lower** value = > better performance
- **Throughput** (or Bandwidth)
  - amount of work performed per unit of time (e.g., number of FP operations per sec.)
  - **higher** value = > better performance
- In some cases:
  - Throughput = 1/Latency
- Examples of bandwidth:
  - **Network** (Fiber Internet): 500 Mbit/s
  - **Memory** (Apple M3): 100 GByte/s
  - **GPU** (NVIDIA) RTX 5060): 20 TFLOP/s
- Latency vs bandwidth
  - Fiber Internet 2 Gbit/s has 4x less latency?

Os sistemas de computação atuais apresentam um elevado ***débito*** [de instruções]



Technological evolution has provided an improvement in the **throughput** of the various computer components, while **latency** has improved at a much slower rate

Revisão da última aula (3)

of Computer Design

- The Processor Performance Equation
  - A simple **model** that relates high-level performance metric (CPU time) with several low-level **metrics**

$$CPU\ time = Instruction\ count \times Cycles\ per\ instruction \times Clock\ cycle\ time$$

Our notation:

Texe = #I x CPI x Tcc

$$CPI = \frac{CPU\ clock\ cycles\ for\ a\ program}{Instruction\ count}$$

- Example

```
   movl 10, %eax
   movl 0, %ecx
loop:
   addl %eax, %ecx
   decl %eax
   jnz loop
```

Estimate execution time in a computer system with a 2 GHz clock frequency and a CPI of 1.5

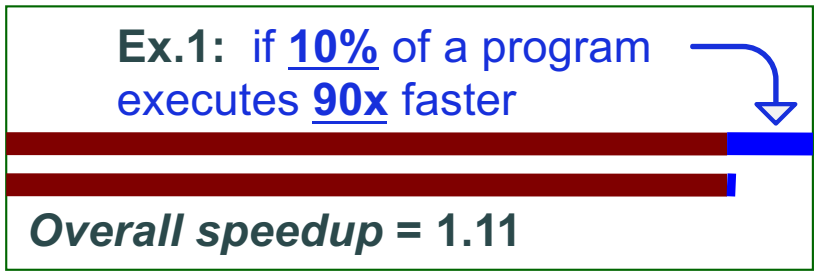
CPU_{time} = 32 x 1.5 x 1/(2x10^{-9}) sec

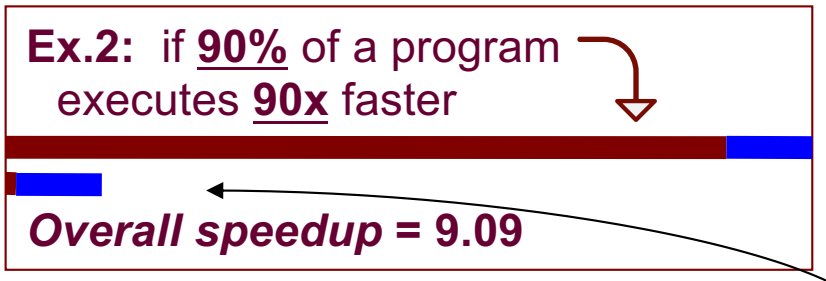
= 24 * 10^{-9} s = **24 ns**

Copyright © 2019, Elsevier Inc. All rights reserved.

Measuring performance improvements

- **Speedup** (or gain) of X relative to Y
  - Execution time$_Y$ / Execution time$_X$
  - How to measure the execution time?
    - Wall clock time: time from start to finish - includes all system overheads
    - CPU time: includes only computation time – hard to measure!
- **Impact of performance improvements**: Amdahl law's
  - Speedup is limited by the program fraction that is not improved





  - **Corollary**: focus on program execution **Hot-spots**
    - **Parts of the program that take most execution time**
    - **Note: hot-spots might change as the program is being optimized**

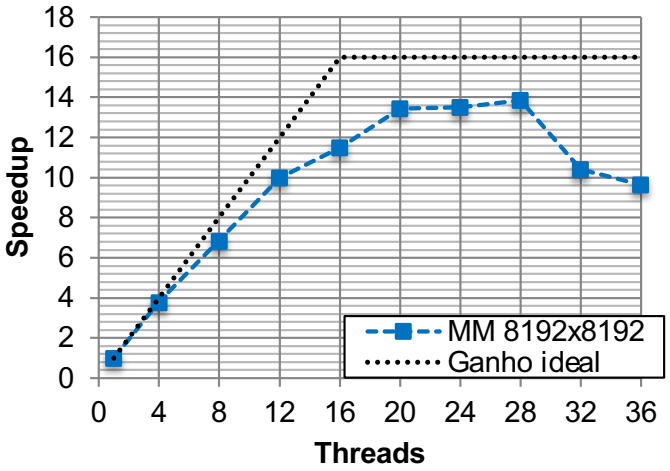
Parallel speedup

speedup (gain)

- execution time of the best sequential implementation divided by the execution time of the parallel version

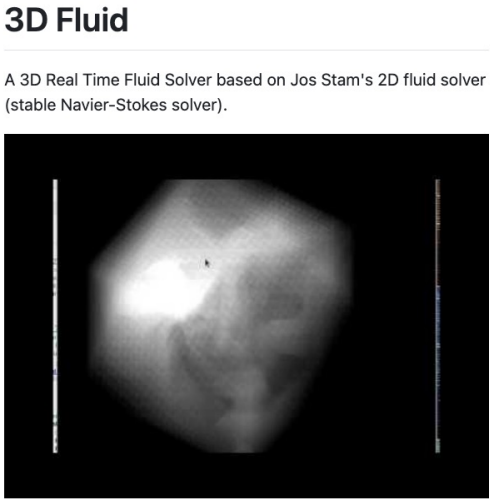
[traditional] scalability analysis

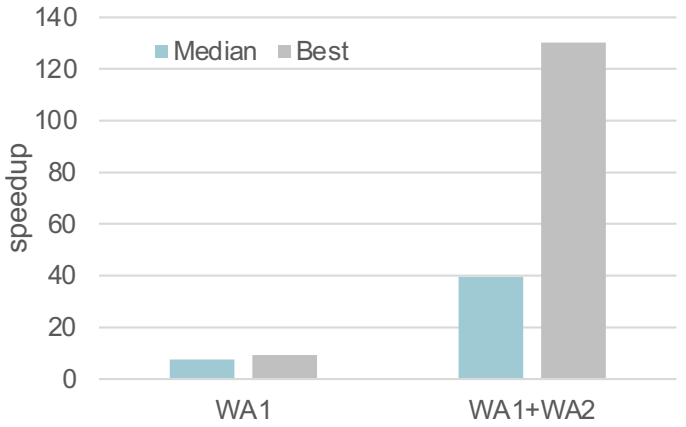
- Speedup increase with the number of
- Processing Units (PUs)



130-fold speedups

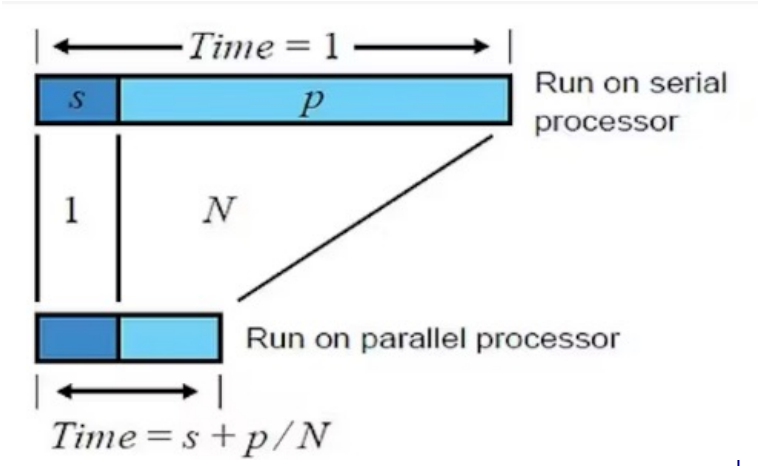
On a 20-core system?





Performance improvement limit: Serial part

- **Amdahl law's & parallelism**



P = faction enhanced (or parallel fraction; 1 = 100%)

$$\text{Speedup}(N) = \frac{1}{(1-P)+\frac{P}{N}}$$

Serial part of job = 1 (100%) - Parallel part

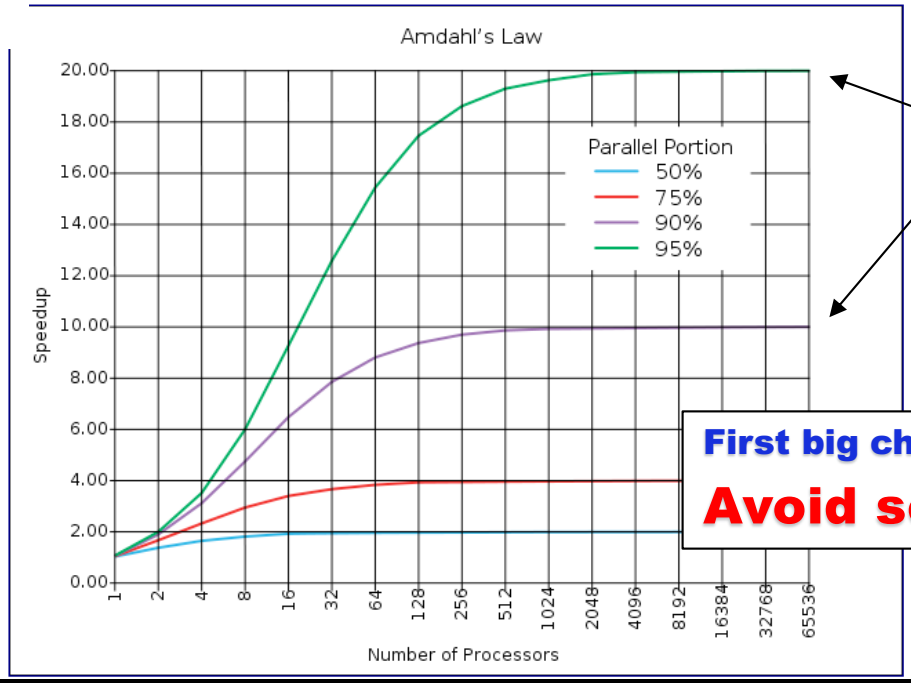
Parallel part is divided up by N workers

If *parallel part* is 95%

$\text{Speedup}_{\text{limit}} =$

$$\frac{1}{(1-0.95)+\frac{0.95}{N}} =$$

$$\frac{1}{0.05+\frac{0.95}{\infty}} = 20$$



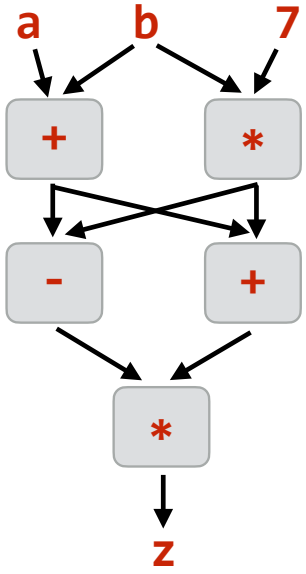
Serial part limitations

First big challenge:
Avoid serial parts

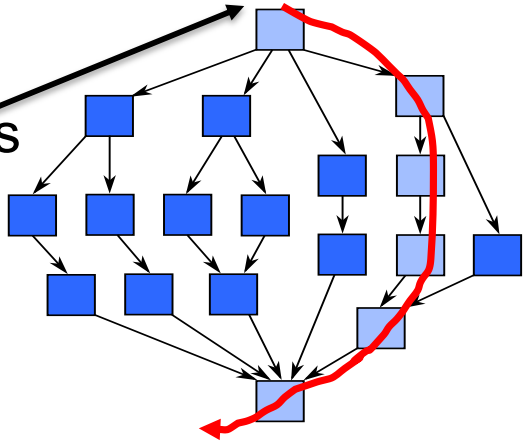
Parallelism: key concepts

- Understanding **dependencies** is a key
  - The lack of dependencies implies potential parallelism

```
x = a + b;
y = b * 7;
z = (x-y) * (x+y);
```



- Program dependency graph
  - Arrows represent dependencies
  - Boxes represent “tasks”
  - **Critical path**: longest chain



- A [time] **schedule** maps tasks to compute resources respecting program dependencies
  - The critical path is the intrinsic execution time limitation
  - There can be many possible schedules on restricted resources

Parallelism: key concepts

- Task **granularity***: What is a ”task”?
  - A single or multiple instructions (e.g., Instruction Level Parallelism)
  - One or several program statements
  - Call to functions of subroutines
  - A larger part of a program
  - A complete program (e.g., workflow)



*) A granule is a small particle of a substance, like a granule of sugar

- Types of **dependencies** between tasks
  - **Data**: data produced by a task is necessary for the dependent task
    - `c = a + b`
  - **Control**: execution of a task is dependent of a previous tasks
    - `if (cond) a;`

Parallelism: key concepts

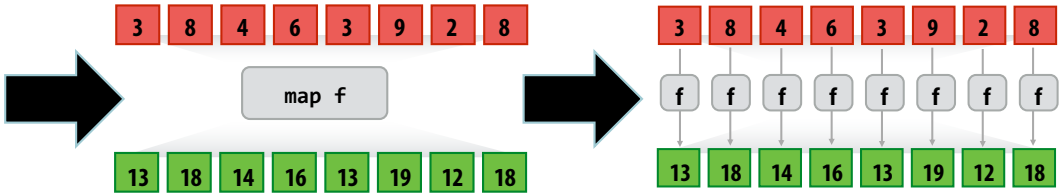
- Strategies to explore parallelism
  - **Function parallelism** - runs different program functions in parallel
    - Speedup is limited by the number of task (i.e., functions)
  - **Data parallelism** – runs the same function across a set of data
    - Better: Parallelism scales as the data size increases

- Common **data parallelism** patterns
  - **Map**: applies side-effect free unary function to all input elements

C++

```
int f(int x) { return x + 10; }

int a[] = {3, 8, 4, 6, 3, 9, 2, 8};
int b[8];
std::transform(a, a+8, b, f);
```



  - **Reduce**: Applies a binary operation returning an accumulated value

